StockManager

Sistema de Gerenciamento de Estoque

Aplicação Full Stack desenvolvida para gerenciar produtos, categorias e movimentações de estoque. O sistema permite cadastrar e editar produtos, controlar entradas e saídas e consultar o histórico de movimentações.

Tecnologias utilizadas

Backend

- Java 26
- Spring Boot
- Spring Web
- Spring Data JPA
- Hibernate
- MySQL
- Gradle
- JUnit 5 e Mockito

Frontend

- React
- JavaScript
- Vite
- HTML e CSS
- Fetch API

Funcionalidades

- Cadastro e consulta de categorias.
- Cadastro, consulta e edição de produtos.
- Associação de produtos às categorias.
- Registro de entradas de estoque.
- Registro de saídas de estoque.
- Consulta do histórico de movimentações.
- Atualização automática da quantidade em estoque.
- Tratamento de erros e validações.

Regras de negócio

1. O nome da categoria não pode ser vazio ou duplicado.
2. O código do produto deve ser único.
3. O preço do produto deve ser maior que zero.
4. A quantidade em estoque e o estoque mínimo não podem ser negativos.

5. Não é permitido registrar movimentações com quantidade menor ou igual a zero.
6. Não é permitido realizar uma saída maior que a quantidade disponível em estoque.
7. Toda movimentação registra o tipo, a quantidade, a data e o produto relacionado.

Banco de dados

O projeto utiliza MySQL para persistência dos dados.

As principais entidades são:

- **Categoria:** armazena as categorias dos produtos.
- **Produto:** armazena nome, código, preço, estoque e categoria.
- **MovimentacaoEstoque:** registra entradas e saídas dos produtos.

Relacionamentos

- Uma categoria pode possuir vários produtos.
- Um produto pertence a uma categoria.
- Um produto pode possuir várias movimentações de estoque.

O relacionamento entre as entidades é representado no código utilizando JPA, com `@OneToMany` e `@ManyToOne`.

O arquivo `schema.sql`, localizado em `backend/demo/src/main/resources/database/`, contém a estrutura de criação das tabelas, chaves primárias, chaves estrangeiras e restrições.

Endpoints da API

A API utiliza o endereço base:

`http://localhost:8080`

Categorias

Método	Endpoint	Descrição
POST	`/api/categorias`	Cadastrar categoria
GET	`/api/categorias`	Listar categorias
GET	`/api/categorias/{id}`	Buscar categoria por ID

Produtos

Método	Endpoint	Descrição
POST	`/api/produtos`	Cadastrar produto
GET	`/api/produtos`	Listar produtos
GET	`/api/produtos/{id}`	Buscar produto por ID

Método	Endpoint	Descrição
GET	`/api/produtos/categoria/{categoriaId}`	Listar produtos por categoria
PUT	`/api/produtos/{id}`	Atualizar produto

Movimentações

Método	Endpoint	Descrição
POST	`/api/movimentacoes`	Registrar movimentação
GET	`/api/movimentacoes`	Listar movimentações
GET	`/api/movimentacoes/produto/{produtoId}`	Consultar movimentações por produto

Como executar o projeto

Pré-requisitos

- Java JDK 26 instalado.
- MySQL instalado e em execução.
- Node.js e npm instalados.
- Git instalado.

1. Configurar o banco de dados

Crie o banco de dados no MySQL:

```
CREATE DATABASE stockmanager;
```

Configure o usuário MySQL utilizado pela aplicação. O projeto utiliza o usuário `stockmanager_app`.

A senha deve ser configurada por meio da variável de ambiente `DB_PASSWORD`, evitando armazenar credenciais diretamente no código-fonte.

No Windows PowerShell, configure a variável antes de executar o backend:

```
$env:DB_PASSWORD = "sua_senha_do_mysql"
```

Substitua `sua_senha_do_mysql` pela senha configurada para o usuário `stockmanager_app`.

O arquivo de configuração está localizado em:

`backend/demo/src/main/resources/application.properties`

A conexão utiliza a variável `${DB_PASSWORD}` para acessar a senha com segurança.

O arquivo `schema.sql` está configurado para ser executado automaticamente durante a inicialização do backend.

O Hibernate também está configurado para atualizar o mapeamento das entidades conforme a configuração do projeto.

Importante: nunca publique senhas reais ou credenciais pessoais no repositório.

2. Executar o backend

Abra um terminal na pasta principal do projeto e execute:

```
cd backend/demo
```

Se a variável `DB_PASSWORD` ainda não estiver configurada no terminal, configure-a conforme explicado na seção anterior.

Inicie a aplicação:

```
.\gradlew.bat bootRun
```

O backend ficará disponível em:

`http://localhost:8080`

3. Executar o frontend

Abra outro terminal na pasta principal do projeto:

```
cd frontend
```

Instale as dependências:

```
npm install
```

Inicie a aplicação:

```
npm run dev
```

O Vite exibirá o endereço local para acessar o sistema, normalmente:

`http://localhost:5173`

Mantenha os dois terminais abertos enquanto estiver utilizando a aplicação.

Testes automatizados

O backend possui testes automatizados utilizando JUnit 5 e Mockito.

Os testes verificam regras de negócio relacionadas a:

- Cadastro e atualização de produtos.
- Validação de preço.
- Validação de categoria.
- Verificação de código duplicado.
- Entrada de estoque.
- Saída de estoque.
- Bloqueio de saída quando não há estoque suficiente.

Para executar os testes, entre na pasta `backend/demo` e utilize:

```
.\gradlew.bat clean test
```

Estrutura do projeto

```
StockManager/
├── backend/
│   └── demo/
│       ├── src/
│       │   ├── main/
│       │   │   ├── java/
│       │   │   │   └── com/example/demo/
│       │   │   │       ├── config/
│       │   │   │       ├── controller/
│       │   │   │       ├── domain/
│       │   │   │       ├── dto/
│       │   │   │       ├── exception/
│       │   │   │       ├── repository/
│       │   │   │       └── service/
│       │   │   └── resources/
│       │   │       └── database/
│       │   │           └── schema.sql
│       │   └── test/
│       ├── build.gradle
│       └── gradlew.bat
├── frontend/
│   ├── src/
│   │   ├── services/
│   │   ├── App.jsx
│   │   └── App.css
```

```
│   └── package.json
└── README.md
```

Dificuldades Encontradas e Soluções

Durante o desenvolvimento do StockManager, foram encontradas algumas dificuldades técnicas que exigiram testes, ajustes e pesquisas para serem resolvidas.

1. Configuração do ambiente

Uma das primeiras dificuldades foi configurar corretamente o ambiente de desenvolvimento, incluindo Java, Gradle, Spring Boot, Node.js e React.

Solução: foram realizadas configurações e testes individuais para garantir que o backend e o frontend executassem corretamente.

2. Desenvolvimento e integração do backend

Durante a implementação dos endpoints e das regras de negócio, ocorreram erros de compilação e ajustes necessários nos métodos dos serviços.

Solução: os erros foram identificados por meio das mensagens do Gradle e corrigidos nos métodos e nas classes responsáveis pelas operações.

3. Configuração do banco de dados

Foi necessário configurar a conexão entre o Spring Boot e o MySQL, além de criar as tabelas e seus relacionamentos.

Solução: foi configurada a conexão com o MySQL utilizando Spring Data JPA e Hibernate, além da criação do arquivo `schema.sql`, responsável por representar a estrutura inicial do banco de dados.

4. Integração entre React e Spring Boot

Outra dificuldade foi fazer o frontend consumir corretamente os endpoints da API, garantindo que os dados cadastrados fossem exibidos e atualizados na interface.

Solução: foi criado um arquivo de serviços para centralizar as requisições HTTP, utilizando `fetch` para comunicação com a API REST.

5. Validação das operações de estoque

Foi necessário garantir que as movimentações respeitassem as regras de negócio, principalmente para impedir saídas com quantidade superior ao estoque disponível.

Solução: as validações foram implementadas na camada de serviço do backend, retornando mensagens de erro quando uma operação não pode ser realizada.

Considerações sobre as dificuldades

As dificuldades encontradas contribuíram para o aprendizado sobre desenvolvimento Full Stack, integração entre tecnologias, persistência de dados, organização em camadas, tratamento de erros e aplicação de regras de negócio.

Considerações finais

O StockManager foi desenvolvido para demonstrar a integração entre backend e frontend em uma aplicação Full Stack. O projeto utiliza uma API REST com Spring Boot, persistência de dados com JPA e MySQL e uma interface construída com React.

A aplicação aplica regras de negócio para manter a consistência do estoque e permite acompanhar as movimentações realizadas nos produtos.

Nome: joão Guilherme Mello , Luiz Henrique Franco.